MINDMETER · SENIOR PROJECT · DTETI UGM

Frontend + Backend Integration Guide

Everything both sides must agree on,
so the merge just works.

Who reads what

- **Christania** — all of it: pages 2–15
- **Nalen** — pages 2–8 and 14–15
- **Arya** — pages 9–10: the states to design

Tested, not just written

- Backend: 50/50 tests, incl. real responses checked against shared/contract.ts
- Reference frontend code (pages 11–13): strict TypeScript + 11/11 checks against the live API
- Every JSON example is real server output

THE WHOLE PICTURE

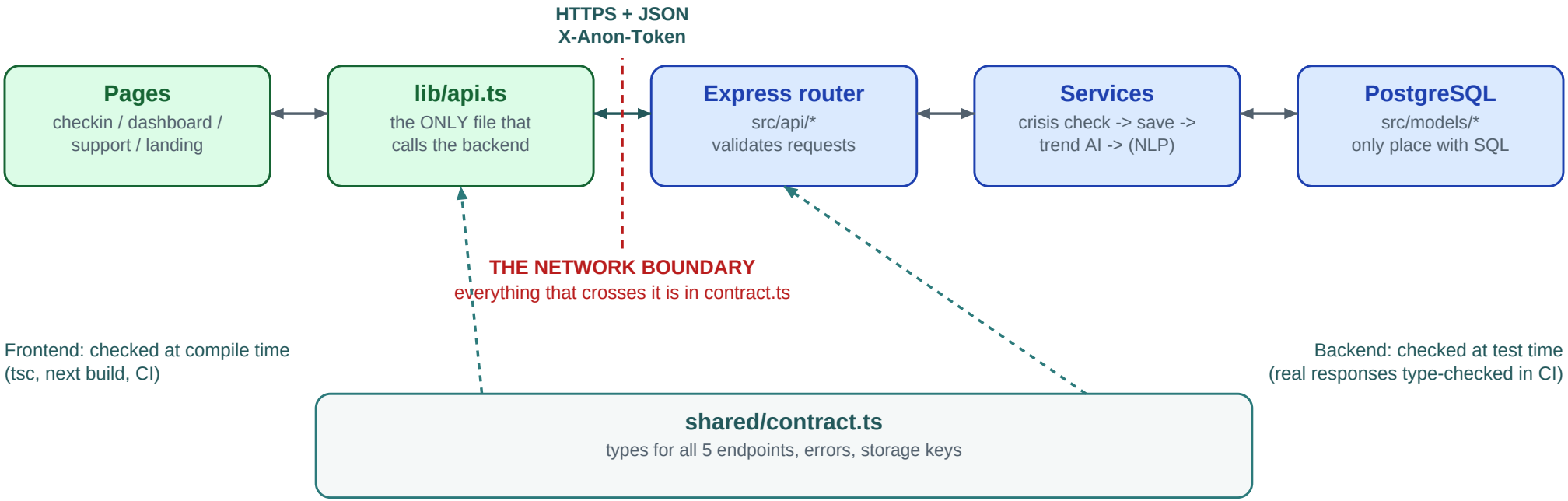
Seven things both sides must agree on

If any one of these differs between frontend and backend, something breaks at merge time. Everything else in this guide expands one of these rows.

Connection	Frontend does	Backend does	What breaks if they disagree
1 API contract	Imports types from `shared/contract.ts`	Integration test checks every real response against it	Wrong field name, missing field, crash on undefined
2 Base URL	`NEXT_PUBLIC_API_URL` (build-time env)	Listens on `PORT`; App Service URL on Azure	Every call fails: "could not reach server"
3 CORS	Runs on its own origin (localhost:3001 / SWA URL)	`CORS_ORIGIN` lists that exact origin	Browser blocks the response (CORS error in console)
4 Identity	Sends `X-Anon-Token` = UUID from localStorage	Hashes it, finds or creates the user	401 ANON_TOKEN_REQUIRED; dashboard empty
5 JSON + errors	Sends JSON; reads `{ error: { code, message } }`	Always answers in that shape	Error screens show "undefined"
6 Storage keys	`mindmeter_anon_token`, `mindmeter_sync_queue`	— (backend never sees localStorage)	Student "loses" history; queue never sent
7 Crisis signal	`crisis_flag: true` -> support screen, nothing else	Decides crisis_flag (keywords, before any AI)	Student in crisis sees a cheerful insight

ARCHITECTURE

Where the frontend and backend touch



Rule: no component calls `fetch`, and nothing on the frontend touches the database. If the API ever changes, there is exactly one file per side to update, and CI proves they still match.

CONFIGURATION

Settings on both sides, locally and on Azure

Side	Setting	Local	Azure	Why it matters
Frontend	`NEXT_PUBLIC_API_URL`	`http://localhost:3000`	`https://<api>.azurewebsites.net`	Where lib/api.ts sends requests. Baked in at build time.
Frontend	`dev port`	`3001 (next dev -p 3001)`	—	Next.js and the API both default to 3000: they collide.
Frontend	`next.config`	`output: 'export'`	`same`	Static site for Azure Static Web Apps; add images: { unoptimized: true }.
Backend	`PORT`	`3000`	`set by Azure`	Where the API listens.
Backend	`CORS_ORIGIN`	`http://localhost:3001 (or empty)`	`https://<site>.azurestaticapps.net`	Must equal the frontend's origin exactly. Required in production.
Backend	`DATABASE_URL / DB_SSL`	`local Postgres / false`	`Azure Postgres / true`	Azure PostgreSQL requires SSL.
Backend	`OPENROUTER_API_KEY / _MODEL`	`empty (NLP off)`	`set in Stage 2`	Never in the frontend.
Backend	`DEMO_ANON_TOKEN`	`any 16+ char token`	`same value`	Demo account; paste into localStorage for the demo.

Run the backend locally (Nalen's side)

```
cd backend && npm install
cp .env.example .env   # set DATABASE_URL, DB_SSL=false
npm run db:migrate && npm run db:seed-demo
npm run dev             # -> http://localhost:3000
```

Run the frontend against it (Christania's side)

```
cd frontend && npm install
echo NEXT_PUBLIC_API_URL=http://localhost:3000 > .env.local
npm run dev -- -p 3001  # -> http://localhost:3001
# check: http://localhost:3000/health says ok
```

THE CONTRACT

shared/contract.ts — the core types

```
export type MoodValue = 1 | 2 | 3 | 4 | 5;
export type Trend = 'stable' | 'declining' | 'improving';
export type ThemeCode = 'sleep' | 'workload' | 'exams'
  | 'isolation' | 'relationships' | 'finance'
  | 'health' | 'motivation';

export interface CheckinRequest {
  mood_value: MoodValue;
  note_text?: string | null;     // max 200 chars
  client_ts?: IsoDateTime | null;
}

export interface CheckinSaved {
  id: Uuid; saved: true;
  insight: string;
  crisis_flag: false;
}
export interface CheckinCrisis {
  id: Uuid; saved: true;
  crisis_flag: true;             // NO insight
}
export type CheckinResponse = CheckinSaved | CheckinCrisis;
```

```
export interface SyncItem {
  mood_value: MoodValue; note_text?: string | null;
  client_ts: IsoDateTime;        // required offline
}
export interface SyncResult {
  client_ts: IsoDateTime; id: Uuid;
  saved: boolean; crisis_flag: boolean;
}

export interface InsightResponse {
  trend: Trend; message: string;
  themes: ThemeCode[]; resources: Resource[];
}
export interface Resource {
  id: Uuid; title: string; description: string;
  url: string;                   // https://... or tel:+62...
  tags: ThemeCode[];
}
export interface ApiErrorResponse {
  error: { code: ApiErrorCode; message: string;
           details?: { index?: number } };
}
```

Why `CheckinResponse` is a union: TypeScript will not let the frontend read `insight` until it has checked `crisis_flag`. The crisis case cannot be forgotten. Full file (173 lines): `shared/contract.ts`.

REAL EXAMPLES · 1

Sending check-ins

POST /api/checkins — normal note

```
Request   X-Anon-Token: 3f0c...-a1b2
{ "mood_value": 3,
  "note_text": "capek tapi oke, tugas numpuk" }

200 OK
{ "id": "44e70f7a-af44-4586-a6c2-55b60e265129",
  "saved": true,
  "insight": "Mood kamu cenderung membaik minggu ini.",
  "crisis_flag": false }
```

POST /api/checkins — crisis note

```
Request   X-Anon-Token: 3f0c...-a1b2
{ "mood_value": 1,
  "note_text": "Capek banget, rasanya ingin mati saja." }

200 OK
{ "id": "44e70f7a-af44-4586-a6c2-55b60e265129",
  "saved": true,
  "crisis_flag": true }          <- no insight: show support
```

POST /api/checkins/sync — queued offline check-ins

```
Request (1-50 items, same order back)
[ { "mood_value": 4, "note_text": "offline dulu",
    "client_ts": "2026-09-26T20:00:00+07:00" },
  { "mood_value": 2,
    "client_ts": "2026-09-27T08:00:00+07:00" } ]

200 OK
[ { "client_ts": "2026-09-26T20:00:00+07:00", "id": "…", "saved": true, "crisis_flag": false },
  { "client_ts": "2026-09-27T08:00:00+07:00", "id": "…", "saved": true, "crisis_flag": false } ]
```

REAL EXAMPLES · 2

Reading data for the dashboard and support screen

GET /api/checkins (oldest first, 90 days)

```
[ { "id": "8a2949fe-c193-44ec-b371-db56052d4839",
    "mood_value": 4,
    "note_text": null,
    "created_at": "2026-09-28T13:00:00.000Z" },
  ... ]

[]   <- new student: show the empty state
```

GET /api/resources (NO token needed)

```
[ { "id": "…", "title": "FT UGM Wellbeing Center",
    "description": "Konsultasi psikolog gratis …",
    "url": "tel:+6281391737076",   <- primary, call
    "tags": ["exams", "isolation", "workload"] },
  { "id": "…", "title": "Unit Layanan Kesehatan Mental UGM",
    "url": "https://ugm.ac.id/id/layanan-kesehatan-terpadu/",
    "tags": ["isolation", "relationships"], … } ]
```

GET /api/insights

```
{ "trend": "declining",
  "message": "Mood kamu rendah 3 hari berturut-turut.
              Tidak apa-apa untuk mencari teman bicara.",
  "themes": ["sleep", "workload"],   <- empty until NLP on
  "resources": [ … up to 3 Resource … ] }
```

Any error (4xx / 5xx)

```
400
{ "error": {
    "code": "VALIDATION_ERROR",
    "message": "mood_value must be an integer from 1 to 5." } }
```

ERRORS

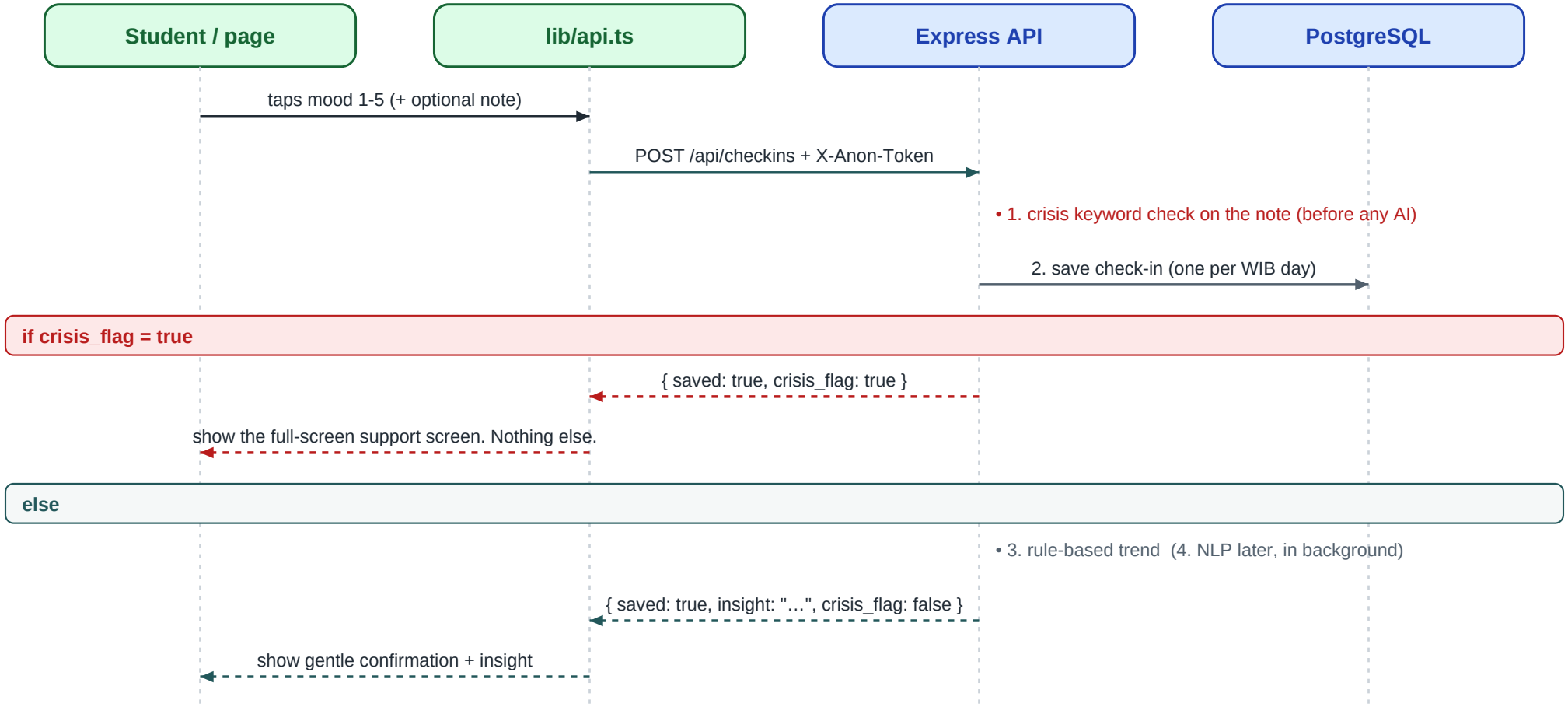
Every error, and what the student should see

error.code	Status	When	What the frontend shows
`NetworkError (no response)`	—	Offline, server down, wrong URL, CORS	Check-in: queue it, show "Tersimpan — akan dikirim saat online". Other screens: gentle retry + cached data if any.
`VALIDATION_ERROR`	400	Bad body (mood 6, note over 200 chars, bad client_ts)	A bug in our frontend: show "Ada yang salah, coba lagi" and log it. The UI should prevent these.
`INVALID_JSON`	400	Body not valid JSON	Bug: same as above.
`ANON_TOKEN_REQUIRED`	401	Missing/invalid X-Anon-Token	Bug in anonToken.ts. Never happens if every call goes through lib/api.ts.
`NOT_FOUND`	404	Wrong path	Bug: path typo. Impossible if paths come from the Endpoints map.
`BODY_TOO_LARGE`	413	Body over 32 kb	Bug: sync sends more than 50 items at once.
`INTERNAL_ERROR`	500	Server or database problem	"Maaf, sedang ada gangguan. Coba lagi sebentar." Keep the Support button visible.

The support screen must never depend on the API succeeding. Keep a hard-coded fallback copy of the primary crisis contact in the frontend, so the Support button still works when the server is unreachable.

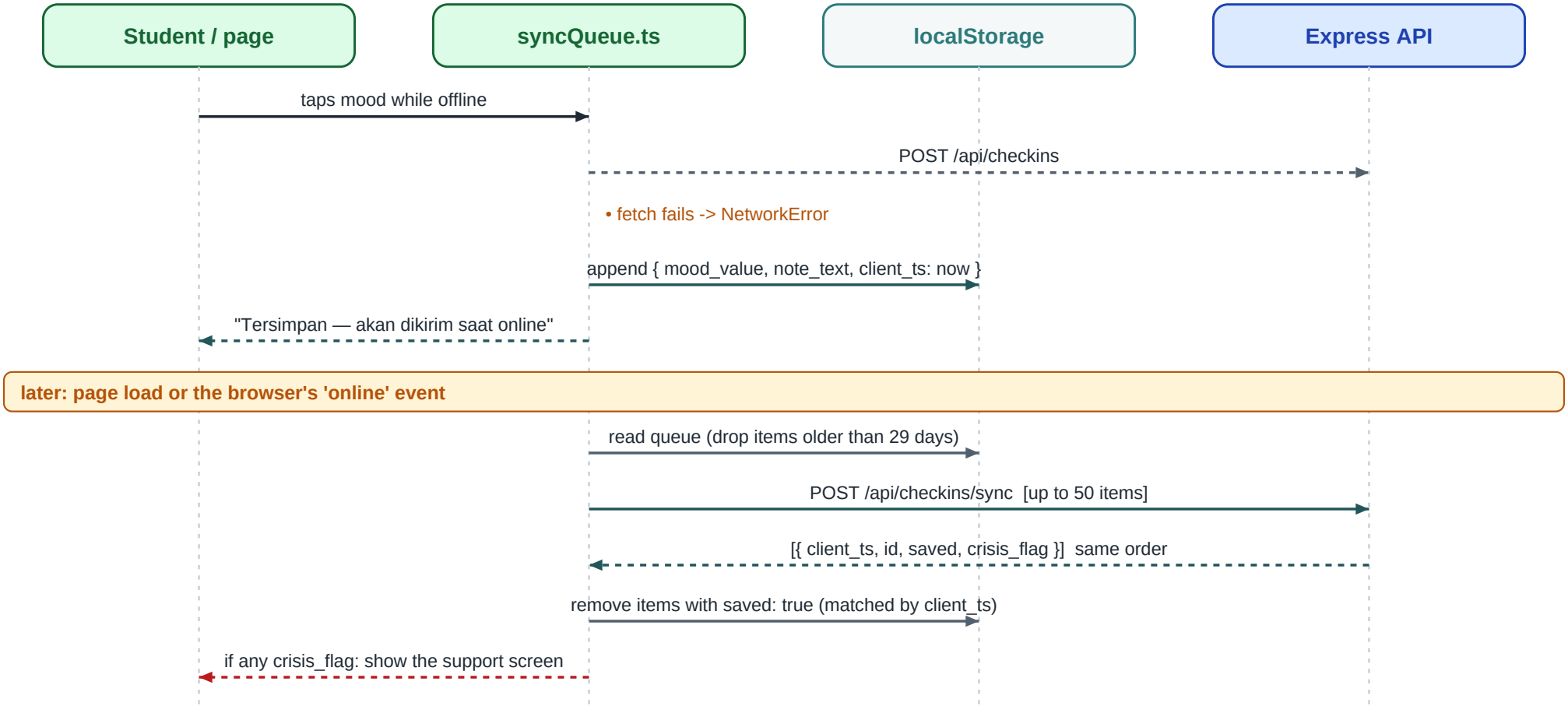
FLOW · 1

A check-in, including the crisis branch



FLOW · 2

Offline check-in and sync (the networking feature)



Re-sending the same queue is safe: the server keeps one check-in per day and the newer `client_ts` wins. Arya: design the "saved offline" state on the check-in screen.

REFERENCE CODE · 1

lib/api.ts + lib/anonToken.ts

```ts
// frontend/src/lib/api.ts
// THE ONLY FILE THAT CALLS THE BACKEND. Components import `api` from here and never call fetch.
// Every request and response type comes from shared/contract.ts, so a wrong field is a compile error.
import type {
  ApiErrorResponse,
  CheckinRequest,
  EndpointKey,
  Endpoints,
  SyncItem,
} from '../../../shared/contract';
import { getAnonToken } from './anonToken';

// Set in frontend/.env.local (local) and in the Static Web Apps build settings (Azure).
// Read inside a function so tests can change it; Next.js still inlines it at build time.
const baseUrl = () => process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3000';

/** The server answered with an error (4xx/5xx). `code` comes from the contract's ApiErrorCode. */
export class ApiError extends Error {
  constructor(
    public readonly status: number,
    public readonly code: string,
    message: string,
    public readonly details?: { index?: number },
  ) {
    super(message);
  }
}

/** The request never reached the server (offline, DNS, CORS). Check-ins go to the offline queue. */
export class NetworkError extends Error {}

async function request<K extends EndpointKey>(
  key: K,
  body?: Endpoints[K]['request'],
): Promise<Endpoints[K]['response']> {
  const [method, path] = key.split(' ');
  const headers: Record<string, string> = { 'Content-Type': 'application/json' };
  if (key !== 'GET /api/resources') headers['X-Anon-Token'] = getAnonToken(); // resources needs no token
  let res: Response;
  try {
    res = await fetch(`${baseUrl()}${path}`, {
      method,
      headers,
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new NetworkError('Could not reach the MindMeter server.');
  }

  const data: unknown = await res.json().catch(() => null);
  if (!res.ok) {
    const err = (data as ApiErrorResponse | null)?.error;
    throw new ApiError(res.status, err?.code ?? 'INTERNAL_ERROR', err?.message ?? `HTTP ${res.status}`, err?.details);
  }
  return data as Endpoints[K]['response'];
}

export const api = {
  submitCheckin: (body: CheckinRequest) => request('POST /api/checkins', body),
  syncCheckins: (items: SyncItem[]) => request('POST /api/checkins/sync', items),
  listCheckins: () => request('GET /api/checkins'),
  getInsights: () => request('GET /api/insights'),
  getResources: () => request('GET /api/resources'),
};

// frontend/src/lib/anonToken.ts
// The student's anonymous identity: a random UUID created once and kept in localStorage.
// The backend stores only a hash of it. Never send it anywhere except the X-Anon-Token header.
import type { StorageKey } from '../../../shared/contract';

const KEY: StorageKey = 'mindmeter_anon_token';

export function getAnonToken(): string {
  let token = localStorage.getItem(KEY);
  if (!token) {
    token = crypto.randomUUID(); // 36 chars, matches the backend's /^[A-Za-z0-9_-]{16,128}$/
    localStorage.setItem(KEY, token);
  }
  return token;
}
```

Copy from docs/integration/reference-lib/ into frontend/src/lib/ after scaffolding. Type-checked in strict mode and run against the live API.

REFERENCE CODE · 2

lib/syncQueue.ts

```
// frontend/src/lib/syncQueue.ts
// Offline check-ins. If the network is down, the check-in is queued in
    localStorage and sent
// later with POST /api/checkins/sync. Re-sending is safe: the server keeps
    the newer tap per day.
import type {
  CheckinRequest,
  CheckinResponse,
  StorageKey,
  SyncItem,
  SyncQueue,
  SyncResponse,
} from '../../../shared/contract';
import { api, ApiError, NetworkError } from './api';

const KEY: StorageKey = 'mindmeter_sync_queue';
const BATCH = 50; // backend limit per sync call
const MAX_AGE_MS = 29 * 24 * 60 * 60 * 1000; // backend rejects items older
    than 30 days

export function readQueue(): SyncQueue {
  try {
    const parsed: unknown = JSON.parse(localStorage.getItem(KEY) ?? '[]');
    return Array.isArray(parsed) ? (parsed as SyncQueue) : [];
  } catch {
    return [];
  }
}

function writeQueue(queue: SyncQueue): void {
  localStorage.setItem(KEY, JSON.stringify(queue));
}

/** What the check-in screen gets back. */
export type SubmitOutcome =
  | { kind: 'saved'; response: CheckinResponse } // check response.crisis_flag
      first!
  | { kind: 'queued' }; // offline: show "Tersimpan — akan dikirim saat online"

/** Use this from the check-in screen instead of api.submitCheckin. */
export async function submitOrQueue(body: CheckinRequest):
    Promise<SubmitOutcome> {
  try {
    return { kind: 'saved', response: await api.submitCheckin(body) };
  } catch (err) {
    if (!(err instanceof NetworkError)) throw err; // a real 4xx/5xx: show the
        error, do not queue
    const item: SyncItem = {
      mood_value: body.mood_value,
      note_text: body.note_text ?? null,
      client_ts: new Date().toISOString(), // when the student actually tapped
    };
    writeQueue([...readQueue(), item]);
    return { kind: 'queued' };
  }
}

let flushing = false;

/**
 * Sends the queue. Returns crisis=true if any queued note was a crisis note:
 * the caller must then show the support screen.
 */
export async function flushQueue(): Promise<{ sent: number; crisis: boolean }>
    {
  if (flushing) return { sent: 0, crisis: false };
  flushing = true;
  let sent = 0;
  let crisis = false;
  try {
    // Drop items the server would refuse for being too old, so the queue can
        never get stuck.
    let queue = readQueue().filter((i) => Date.now() - new
        Date(i.client_ts).getTime() < MAX_AGE_MS);
    writeQueue(queue);

    while (queue.length > 0) {
      const batch = queue.slice(0, BATCH);
      let results: SyncResponse;
      try {
        results = await api.syncCheckins(batch);
      } catch (err) {
        // One invalid item fails the whole batch; the server says which one.
            Drop it and retry.
        if (err instanceof ApiError && err.status === 400 &&
            err.details?.index !== undefined) {
          queue = queue.filter((_, i) => i !== err.details?.index);
          writeQueue(queue);
          continue;
        }
        throw err; // offline again or server error: keep the queue for next
            time
      }
      const done = new Set(results.filter((r) => r.saved).map((r) =>
          r.client_ts));
      crisis = crisis || results.some((r) => r.crisis_flag);
```

Check-in screen: use submitOrQueue(). Root layout: call startAutoSync(() => router.push('/support')) once.

REFERENCE CODE · 2 (CONT.)

lib/syncQueue.ts (continued)

```
      sent += done.size;
      queue = readQueue().filter((i) => !done.has(i.client_ts));
      writeQueue(queue);
    }
  } catch (err) {
    if (!(err instanceof NetworkError)) console.error('[sync]', err);
  } finally {
    flushing = false;
  }
  return { sent, crisis };
}

/** Call once from the root layout: syncs on page load and whenever the
    browser comes back online. */
export function startAutoSync(onCrisis: () => void): () => void {
  const run = () => flushQueue().then((r) => r.crisis && onCrisis());
  run();
  window.addEventListener('online', run);
  return () => window.removeEventListener('online', run);
}
```

Check-in screen: use submitOrQueue(). Root layout: call startAutoSync(() => router.push('/support')) once.

MERGING SAFELY

Gates, the contract-change rule, and the PR checklist

CI gates on every PR

Backend: ESLint + 50 tests on a real PostgreSQL, including the contract check.

Frontend: ESLint + `tsc --noEmit` + `next build` (Christania adds this job in session 6).

Runs when `backend/**`, `frontend/**` or `shared/**` change.

Red CI = no merge. No exceptions before a demo.

Changing the API

1. Open an issue describing the change, and tag Nalen and Christania.

2. One PR changes `shared/contract.ts` + backend code + backend tests.

3. Label it **contract-change**; both must approve.

4. After it merges, the frontend fixes any new type errors in the next PR.

Adding a field is safe; renaming or removing one breaks the other side.

PR checklist

[ ] Branch is up to date with `dev`.

[ ] CI green.

[ ] Only `lib/api.ts` calls the backend.

[ ] Loading, empty, error, offline, crisis states handled.

[ ] No secrets (`.env`, API keys, demo token) committed.

[ ] Tried it against the real API (local or Azure).

[ ] 1 teammate approved.

TROUBLESHOOTING

When the frontend and backend don't talk

Symptom	Likely cause	Fix
Console: "blocked by CORS policy"	CORS_ORIGIN doesn't match the frontend origin (port included)	Set CORS_ORIGIN=http://localhost:3001 (local) or the Static Web Apps URL; restart the API.
Every call: NetworkError	API not running, wrong NEXT_PUBLIC_API_URL, or port clash on 3000	Open <API_URL>/health. Run Next.js on 3001. Rebuild after changing .env.local.
401 ANON_TOKEN_REQUIRED	Call made outside lib/api.ts, or localStorage token corrupted	Route every call through lib/api.ts; clear mindmeter_anon_token and reload.
Dashboard empty for the demo	Browser is using a new anonymous token	localStorage.setItem('mindmeter_anon_token', DEMO_ANON_TOKEN), then reload.
Crisis note does not trigger support	crisis_keywords table empty (server logs a warning)	Review data/crisis-keywords.json, then npm run db:seed-safety.
Support screen shows no contacts	resources not loaded, or not verified	Set verified_at in data/resources.json, then npm run db:seed-safety.
Type error after pulling dev	contract.ts changed in a contract-change PR	Fix the frontend to match the new types; that is the system working.
Insight themes always []	NLP (Stage 2) is off	Expected in Stage 1. Set OPENROUTER_API_KEY + OPENROUTER_MODEL for Stage 2.